

Hawkeye Replacement Policy

High Performance Computer Architecture

Assignment-01 Report

Karan Suthar

SR No.: **28327**

1 Observations and Learnings

1.1 Why is the history size $8W$ and not $16W$ or $32W$?

One thing which I initially did not understand in the implementation was why the history length is taken as

$$\text{History Length} = 8W,$$

where, W = associativity of cache set

The reason comes from the way Hawkeye tries to reconstruct the behaviour of Belady's optimal replacement policy. Belady's algorithm can look arbitrarily far into the future before deciding which cache line should be removed. Since Hawkeye cannot actually see the future, OPTgen instead looks at a sufficiently long history of previous accesses.

The paper experimentally studies how much information is required. It shows that increasing the observation window improves the accuracy of the reconstructed OPT decisions, but after around $8\times$ the cache capacity the result becomes very close to the full OPT solution. So $8\times$ gives a good balance between accuracy and the amount of history that needs to be stored.

For a set-associative cache, OPT decisions are independent for each set. Therefore, for a W -way set, the capacity of that set is W cache lines and the required history becomes approximately

$$8 \times W.$$

For example, in 16-way LLC, $8W = 8 \times 16 = 128$ history positions are maintained for 1 set.

We could theoretically use $16W$ or $32W$, but this would require more storage and more history management. According to the paper, most of the useful accuracy has already been obtained at approximately $8W$. Therefore, increasing the history further gives much less benefit compared to the additional implementation and storage cost.

This helped me understand that the value 8 is not an arbitrary constant in the code; it comes from the experimental study of how much history is needed to approximate OPT accurately.

1.2 Why does the Hawkeye Predictor have 8192 entries?

Another implementation detail which I found interesting was the size of the Hawkeye Predictor. The predictor contains

$$8192 = 2^{13}$$

entries, and every entry is a 3-bit saturating counter.

The predictor is indexed using a 13-bit hashed value of the program counter (PC). Therefore, a 13-bit index can directly select one of $2^{13} = 8192$ predictor entries.

Each entry learns whether accesses generated by the corresponding hashed PC usually behave as cache-friendly or cache-averse according to OPTgen. If OPTgen reports an OPT hit, the counter moves towards the cache-friendly state. If OPTgen reports an OPT miss, the counter moves towards the cache-averse state.

The interesting part is the hardware tradeoff. We would ideally like to keep separate information for every load instruction, but this would require a very large table. Instead, Hawkeye uses a fixed-size hashed table.

The storage required for this predictor is approximately

$$8192 \times 3 = 24576 \text{ bits}$$

$$\frac{24576}{8} = 3072 \text{ bytes} \approx 3 \text{ KB}.$$

The paper therefore gets a reasonably large PC-indexed predictor using only about 3 KB of storage.

If the table was much smaller, more unrelated PCs would map to the same entry and interfere with each other’s training. On the other hand, using 16K or 32K entries would increase the storage cost significantly.

So I understood the 8192-entry table as a practical hardware tradeoff: it is large enough to learn useful per-PC behaviour while still keeping the predictor storage small. The paper uses this 8K-entry predictor as part of Hawkeye’s low-overhead design.

1.3 A long usage interval can still be an OPT hit

Initially, I thought that the length of the usage interval alone should tell us whether a cache line will be useful. For example, if a line is accessed again only after a very long time, it seems natural to think that it should have been removed from the cache.

However, Hawkeye shows that this is not always correct.

Suppose a cache line X is accessed at time t_1 and then accessed again at time t_2 . Its usage interval is

$$[t_1, t_2).$$

Even if this interval is very long, X can still remain in the cache if there is enough cache capacity during that entire interval.

What actually matters is not only how long the reuse interval is, but how many other useful cache lines are competing for the cache during the same time.

For example, consider a 2-way cache. A line may have a long reuse interval, but if only one other live cache line exists during most of that interval, there is still enough space to keep the first line. Therefore, its next access can still be an OPT hit.

On the other hand, another line may have a much shorter reuse interval, but if many liveness intervals overlap during that time and the cache capacity becomes full, OPT may decide not to keep that line.

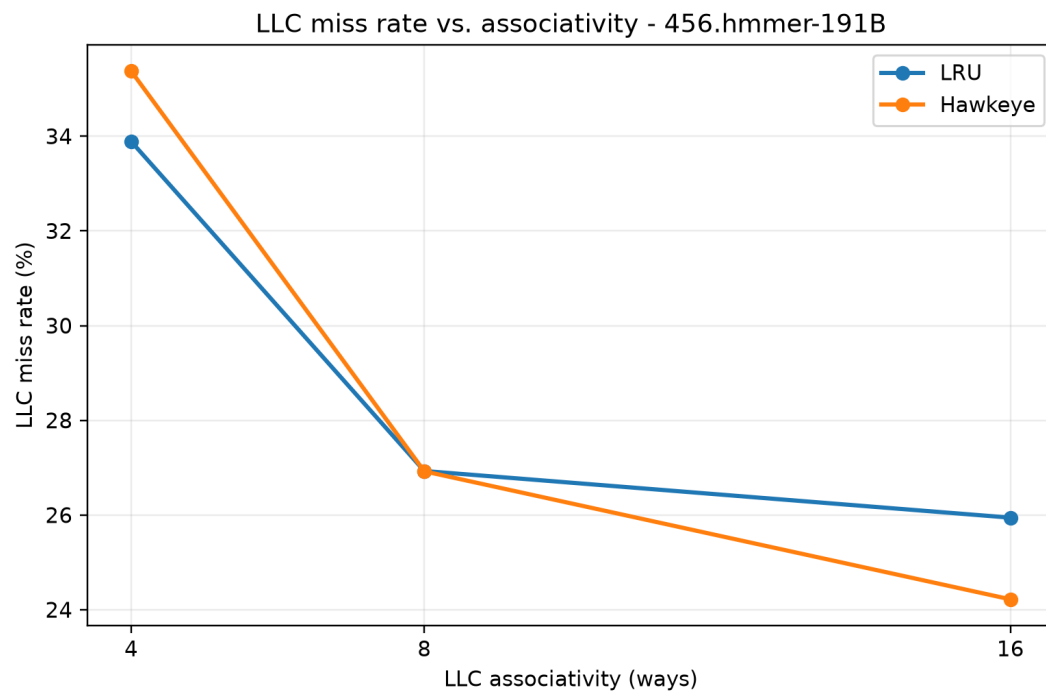
This is why OPTgen maintains an **occupancy vector**. Each entry records how much cache capacity is already required by other liveness intervals.

For a W -way cache, if every occupancy value in the usage interval is less than W , there is enough capacity and the reuse can be classified as an OPT hit. If the occupancy reaches W , there is no free capacity and it becomes an OPT miss.

This observation made it clear to me why simply measuring reuse distance is not sufficient. Hawkeye considers both the reuse interval and the cache demand during that interval.

2 Experimental Results

2.1 Plot 1: LLC Miss Rate vs Associativity



2.2 Plot 2: LLC Miss-Rate Reduction over LRU

